SVEUČILIŠTE U ZAGREBU
Fakultet elektrotehnike i računarstva

Zadatak 2

Igra Connect4, Ivan Džanija 0036547433

Implementacija

Implementacija je napravljena što efikasnije je moguće uz optimizacije poput izbjegavanja kopiranja, korištenja izrazito malih tipova podataka (slanje ploče je 42B, a da nema trećeg stanja bilo bi 42 bita ;( ), ranog izlaska te prepoznavanja nemogućih poteza/smjerova.

```
while (!finished) {
  // Human move
  if (rank == 0 && turn == Turn::HUMAN) {
    board.display();
    i32 move;
    std::cout << "Choose you move (0-6): " << std::endl;
    std::cin >> move;
    for (i32 i = ROWS - 1; i >= 0; --i) {
      if (board.grid[i][move] == BoardCell::EMPTY) {
        board.grid[i][move] = BoardCell::HUMAN;
        board.display();
        if (board.find_winner(i, move) == BoardCell::HUMAN) {
          std::cout << "Victory!" << std::endl;
          finished = true;
        }
        break;
      }
    }
    turn = Turn::COMPUTER;
  }

  // Sync, this makes sure all the processes wait here for the human to make a move
  MPI_Bcast(&finished, 1, MPI_C_BOOL, 0, MPI_COMM_WORLD);
  if (finished) {
    break;
  }
  MPI_Bcast(board.grid.data(), ROWS * COLS, MPI_BYTE, 0, MPI_COMM_WORLD);
```

Gornja snimka zaslona pokazuje kod za odigravanje ljudskog poteza i distribuciju trenutnog stanja ploče. Ukoliko je igrač pobijedio onda se šalje zastavica za završetak i svi procesi završavaju s radom.

```
std::vector<double> local_sums(COLS, 0.0);
std::vector<i32> local_counts(COLS, 0);

for (i32 t_id = rank; t_id < task_count; t_id += size) {
  auto result = execute_task_by_id(t_id, board);
  if (result.has_value()) {
    i32 first_col = t_id % COLS;  // Computers move is always the unit digit
    local_sums[first_col] += result.value();
    local_counts[first_col] += 1;
  }
}
```

Ovdje vidimo kako se dodjeljuju zadaci kojem procesu. Vidimo da se može postaviti proizvoljan broj "task_count" za razinu zrnatosti koju želimo pri stvaranju zadataka. Worker-master arhitektura nije korištena u konačnom rješenju jer je u najboljem slučaju(na samom početku) tek jednaka ovoj implementaciji, a redovito sporija. Testirana je i na puno manjoj zrnatosti i svejedno je ovo rješenje brže. Statistički gledano je zapravo vrlo mala šansa da nam se baš "potrefi" jedan proces koji gotovo stalno ima kratke puteve, dok drugi ima izrazito duge da bi nam se isplatio konstantan "overhead" komunikacije i sinkronizacije. Također jedan proces onda morao ostaviti sa strane za "bookkeeping" što pri broju procesa od <10 je i preko značajnih 10%.

```
Computer played: 2 in: 0.221867s
. . . . . . .
. . . . . . .
. . O O . . .
. O X X . . .
. X O O . . .
X X X O . . .
0 1 2 3 4 5 6

Choose you move (0-6):
0
. . . . . . .
. . . . . . .
. . O O . . .
. O X X . . .
X X O O . . .
X X X O . . .
0 1 2 3 4 5 6

. . . . . . .
. . . . . . .
. . O O . . .
O O X X . . .
X X O O . . .
X X X O . . .
0 1 2 3 4 5 6

Computer played: 0 in: 0.112208s
. . . . . . .
. . . . . . .
. . O O . . .
O O X X . . .
X X O O . . .
X X X O . . .
0 1 2 3 4 5 6

Choose you move (0-6):
4
. . . . . . .
. . . . . . .
. . O O . . .
O O X X . . .
X X O O . . .
X X X O X . .
0 1 2 3 4 5 6

. . . . . . .
. . . . . . .
O . O O . . .
O O X X . . .
X X O O . . .
X X X O X . .
0 1 2 3 4 5 6

Defeat!
Computer played: 0 in: 0.069225s
```

Kvantitativna analiza

Tablica 1. Vrijeme izvođenja[s] za dubinu anglomeracije 1

n = 1	*22.646*	*22.381*	*21.995*	*22.879*	*23.118*

n = 2	*13.157*	*13.357*	*13.312*	*13.594*	*12.736*
n = 3	*10.305*	*10.311*	*10.514*	*10.452*	*10.256*
n = 4	*6.707*	*6.891,*	*6.451,*	*7.154,*	*6.842*
n = 5	*7.528*	*7.634*	*7.312*	*7.891*	*7.427*
n = 6	*7.874*	*7.982*	*7.654*	*8.321*	*7.912*
n = 7	*4.479*	*4.4063,*	*4.529*	*4.396*	*4.673*
n = 8	*4.924*	*4.673*	*3.892*	*5.891*	*4.727*

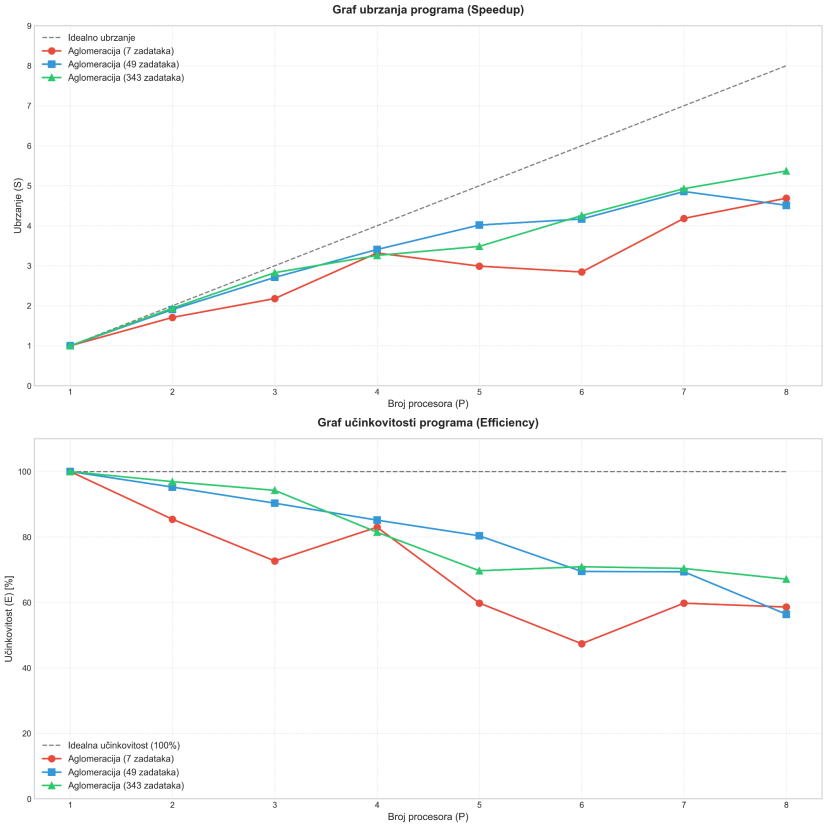
Tablica 2. Vrijeme izvođenja[s] za dubinu anglomeracije 2

n = 1	*22.274*	*22.456*	*22.123*	*22.789*	*22.345*
n = 2	*11.659*	*11.872*	*11.543*	*12.034*	*11.678*
n = 3	*8.169*	*8.345*	*8.012*	*8.567*	*8.234*
n = 4	*6.438*	*6.612*	*6.301*	*6.987*	*6.543*
n = 5	*5.431*	*5.612*	*5.301*	*5.987*	*5.543*
n = 6	*5.221*	*5.412*	*5.101*	*5.787*	*5.343*
n = 7	*4.521*	*4.712*	*4.301*	*4.987*	*4.543*
n = 8	*4.777*	*4.962*	*4.651*	*5.437*	*4.993*

Tablica 3. Vrijeme izvođenja[s] za dubinu anglomeracije 3

n = 1	*22.548*	*22.812*	*22.543*	*23.034*	*22.678*
n = 2	*11.554*	*11.812*	*11.343*	*11.774*	*11.849*
n = 3	*7.928*	*8.112*	*7.812*	*8.387*	*7.943*
n = 4	*6.829*	*7.012*	*6.701*	*7.387*	*6.439*
n = 5	*6.366*	*6.512*	*6.291*	*6.987*	*6.547*
n = 6	*5.061*	*5.212*	*4.993*	*5.587*	*5.143*
n = 7	*4.517*	*4.499*	*4.630*	*4.912*	*4.298*

n = 8	*4.112*	*4.312*	*4.013*	*4.511*	*4.274*



Sva mjerenja su rađena na dubini pretraživanja d = 11.

Na slici iznad vidimo grafove ubrzanja i učinkovitosti. Ako promatramo zelenu krivulju grafa vidimo da je zapravo jako slična izgledu teoretske krivulje. To je zato što su zadaci fino granulirani i raspoređivanje po procesorima jako dobro raspoređeno. Teoretsku liniju nikad ne možemo dosegnuti jer čak i da teoretski nemamo trošak sinkronizacije na mom računalu su 4 snažne jezgre te 4 efikasne jezgre koje rade dosta sporije od snažnih. Dodatna zanimljivost na grafu je primjer crvene krivulje koja predstavlja samo 7 zadataka. Vidimo da za 5 i 6 jezgri ubrzanje je manje nego za 4 jezgre. Razlog tome su upravo ove efikasne jezgre. One na početku uzmu zadatke jer su slobodne, ali bi ih preostale 4 snažne jezgre izvršile brže uz svoje dodatne zadatke. Graf efikasnosti je potpuno očekivan u odnosu na graf ubrzanja te i na njemu vidimo tu pojavu sa 5 i 6 jezgri pri maloj aglomeraciji. Također vidimo da tu pojavu nemamo na većoj granularnosti zadataka.